Воспроизведение статьи «Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates»

О статье

L. N. Smith, N. Topin. Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. arXiv:1708.07120, версия 3 от 2018 года. Код авторов написан на Caffe и лежит на github в `lnsmith54/super-convergence`.

В чём идея

Авторы заметили, что если обучать ResNet-56 на CIFAR-10 с одним циклом learning rate, который поднимается до очень большого значения (3.0 при batch 1000) и опускается обратно, сеть сходится за 10 тысяч итераций к 92.4 %. Обычное обучение с постоянным LR 0.35 и уменьшением в 10 раз на 50 и 70 тысячах итераций даёт 91.2 % за 80 тысяч итераций. Этот эффект они назвали «супер-сходимостью».

Основные тезисы статьи:

1. Перед обучением стоит сделать LR range test: линейно растить LR от нуля и смотреть, когда точность начнёт падать. Для ResNet-56 с batch norm точность не падает даже при LR = 3, то есть на порядок выше обычных значений. Авторы считают это признаком того, что супер-сходимость возможна.
2. Расписание 1cycle: один треугольный цикл LR, который короче всего обучения, а оставшиеся итерации LR линейно уходит на несколько порядков ниже начального. Momentum при этом меняется в противофазе, например 0.95 -> 0.85 -> 0.95.
3. Большой LR сам по себе регуляризует обучение, поэтому остальную регуляризацию нужно ослабить. Weight decay 10^{-3} ломает супер-сходимость, 10^{-4} нет. Большие batch лучше маленьких: 1536 -> 92.1 %, 1000 -> 92.4 %, 512 -> 91.7 %, 256 -> 89.5 %.
4. Из трёх подряд идущих векторов весов можно оценить оптимальный шаг (формула 8 в статье, упрощённый Hessian-free метод):

$$\epsilon^* = \epsilon \frac{\sum |\theta_{i+1} - \theta_i|}{\sum |2\theta_{i+1} - \theta_i - \theta_{i+2}|}$$

   Для ResNet-56 эта оценка даёт 2-6, то есть большие LR оправданы и кривизна в направлении спуска маленькая.
5. Чем меньше данных, тем больше выигрыш: на 10 тысячах примеров CLR даёт 80.6 % против 71.4 % у обычного обучения, на полном датасете разница только 1.2 п.п.
6. Adam супер-сходимости не показывает, Nesterov, AdaDelta и AdaGrad с 1cycle работают.

7. Эффект показан также для CIFAR-100, MNIST, ImageNet и для wide-ResNet, DenseNet, Inception-ResNet-v2.

Как ставились эксперименты у авторов

ResNet-56 для CIFAR: три стадии по 9 блоков с 16/32/64 каналами, при уменьшении разрешения shortcut делается через average pooling и дополнение каналов нулями. Batch norm с moving average fraction 0.95 вместо стандартных 0.999, иначе бегущая статистика не успевает обновиться за короткое обучение. SGD с momentum 0.9, weight decay 10^{-4}, суммарный batch 1000 на 8 GPU. CLR 0.1 -> 3.0 -> 0.1 со stepsize 5000, то есть 10 тысяч итераций всего. Baseline: LR 0.35, деление на 10 на 50 и 70 тысячах, всего 80 тысяч итераций.

На это стоит обратить внимание: 10 тысяч итераций при batch 1000 это 200 эпох, а 80 тысяч это 1600 эпох. Ускорение «на порядок» в статье считается по итерациям при большом batch, а не по эпохам.

Что я воспроизводил

Таблица ниже сопоставляет эксперименты статьи и мои запуски. Имена запусков совпадают с папками в `results/`.

	Статья	Мои запуски
LR range test (Fig. 2b, 8a)	LR до 3 за 5k-100k итераций	LR 0 -> 3 за 10 эпох, batch 128 / 512 / 1000: `range_bs*`
Оценка LR по формуле 8 (Fig. 3a)	первые 300 итераций	то же при LR 0.1 и при CLR: `hf_const`, `hf_clr`
1cycle против step (Fig. 1, Table 6)	10k против 80k итераций	1cycle 25 / 50 / 100 эпох, step 50 / 100 / 200 эпох: `1cycle_e*`, `step_e*`
Мало данных (Table 1, Fig. 4a)	10k-40k примеров	10k и 20k, 1cycle 1500 итераций против step 3000: `sub10k_*`, `sub20k_*`
Weight decay (Fig. 12b)	10^{-3}...10^{-6}	10^{-3} / 10^{-4} / 10^{-5} при 25 эпохах: `1cycle_e25_wd*`
Максимальный LR (Table 6)	1…3.5	0.3 / 1 / 3 при 25 эпохах: `1cycle_e25_lr*`
Adam (Fig. 9)		Adam с 1cycle по LR и β_1: `adam_1cycle_e25`
Generalization gap (Fig. 5b)		train – test по ходу обучения из основных запусков

Чем моя постановка отличается

Всё считалось на MacBook Air M5 (16 ГБ) в PyTorch 2.14 через MPS в fp32. Это накладывает бюджет: одна эпоха ResNet-56 занимает около 35 секунд, 100 эпох это больше часа. Поэтому:

- batch 512 вместо 1000. В статье batch 512 тоже используется (Table 2), и по Table 7 разница с 1000 около 0.7 п.п. В range test я дополнительно прогнал batch 128 и 1000, чтобы увидеть, как от batch зависит допустимый LR;
- длина обучения считается в эпохах, 25-200, а не в сотнях эпох, как у авторов. Честное сравнение тогда такое: 1cycle получает столько же или меньше эпох, чем step;
- вместо чистого треугольника с Fig. 1a я использовал политику 1cycle из той же статьи: 45 % итераций подъём, 45 % спуск, последние 10 % линейное снижение до $\eta_{\min}/100$, momentum 0.95 -> 0.85 -> 0.95;
- аугментация стандартная для CIFAR: случайный сдвиг с padding 4 и горизонтальное отражение. В статье про аугментацию ничего не написано, в caffe-конфигах, на которые они ссылаются, crop и mirror есть;
- инициализация Kaiming, первая свёртка со stride 1. В Table 5 статьи написано stride 2, но это не сходится с average pooling 8×8 в конце, в caffe-конфигах авторов stride 1;
- ImageNet, другие архитектуры, CIFAR-100, MNIST, dropout и свип по momentum я не повторял.

Код

Всё лежит в hw2/. Датасет целиком держится на GPU в uint8, аугментация делается тензорными операциями, так быстрее, чем через DataLoader. `model.py` это ResNet-20/56/110 для CIFAR, `schedules.py` расписания LR и momentum, `train.py` цикл обучения с логом по эпохам и подсчётом оценки по формуле 8, `run_all.sh` очередь из 21 запуска, `plots.py` графики и сводная таблица. По умолчанию SGD, momentum 0.9, weight decay 10^{-4}, batch 512, seed 0, проверка на тесте раз в эпоху (в range test каждые 10 итераций).

Результаты

LR range test

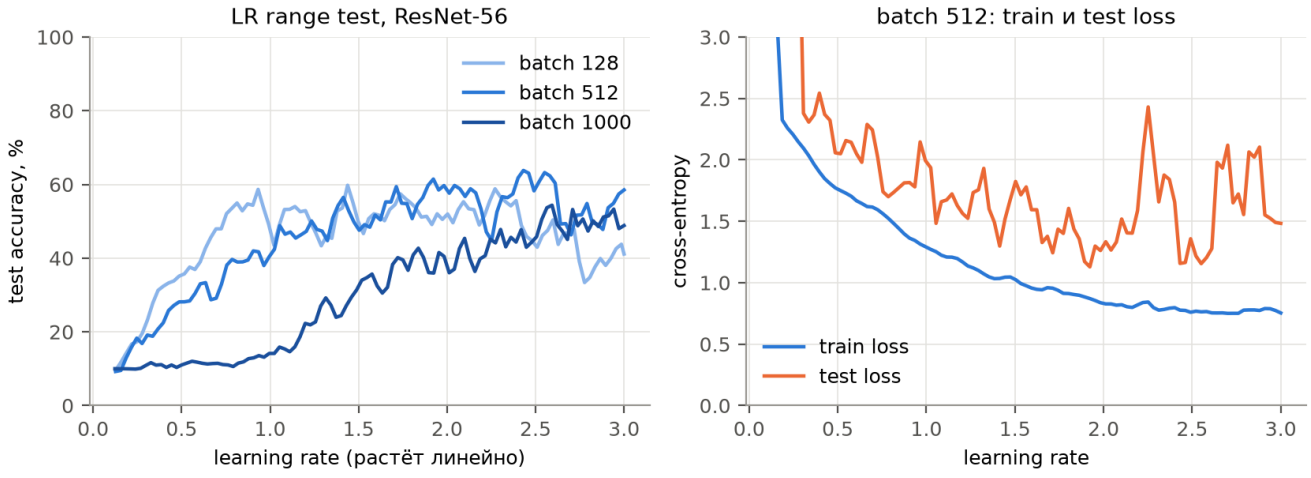


Рис. 1: LR растёт линейно от 0 до 3 за 10 эпох. Слева test accuracy для трёх batch (скользящее среднее по 3 точкам), справа train и test loss при batch 512.

batch	итераций	максимум test acc, % (при каком LR)	средняя test acc при LR ≥ 2.5, %	train acc в конце, %
128	3900	65.8 (2.27)	42.4	67.0
512	970	71.3 (2.43)	54.1	73.9
1000	500	62.6 (2.74)	49.9	67.9

Главное наблюдение статьи подтвердилось: ResNet-56 с batch norm не расходится до LR = 3. Train loss при batch 512 всё время падает, test accuracy после LR ≈ 1 выходит на плато 50-60 % и там шумит, но не обваливается. У авторов на этом тесте точность 80-90 %, но их тест шёл 5-100 тысяч итераций при batch 1000, а у меня 10 эпох, так что уровень плато отличается из-за длины теста.

Два момента, которых в статье нет.

Диапазон допустимых LR зависит от batch. При batch 128 кривая достигает максимума около LR 1-1.5 и к LR = 3 падает до 40 %. При batch 1000 сеть начинает учиться только после LR ≈ 1, потому что за 10 эпох делает всего 500 шагов. Если пересчитать LR = 3 при batch 1000 на batch 128 по линейному правилу, получится LR ≈ 0.4, что вполне обычно. То есть «очень большой LR» из статьи во многом объясняется большим batch.

Признак регуляризации из Fig. 8a не воспроизвёлся. У авторов при LR от 0.3 до 1.5 train loss растёт, а test loss падает. У меня train loss падает монотонно, test loss всегда выше и сильно шумит. Шум я объясняю batch norm с momentum 0.05: при быстро меняющихся весах бегущая статистика не поспевает. Это не опровергает статью, но и не подтверждает трактовку про регуляризацию.

Оценка LR по формуле 8

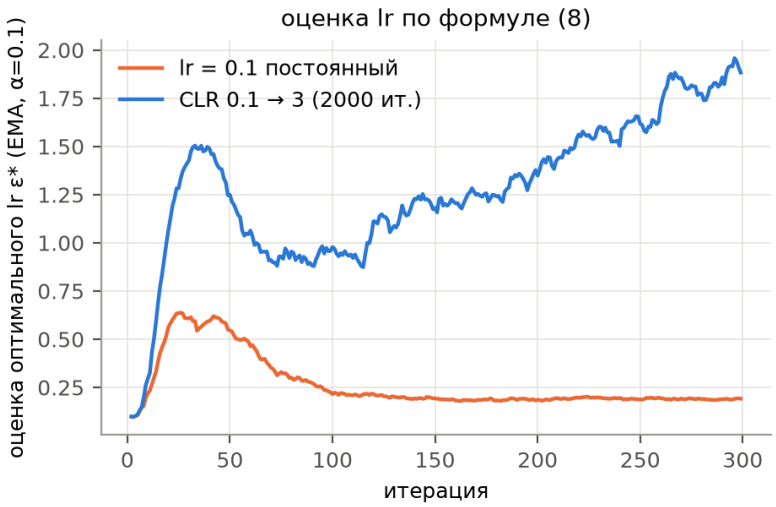


Рис. 2: Оценка оптимального шага по трём подряд идущим векторам весов, EMA с коэффициентом 0.1, первые 300 итераций.

режим	максимум оценки (EMA)	оценка на 300-й итерации
постоянный LR 0.1	0.64 на 26-й итерации	0.19
CLR 0.1 -> 3, 2000 итераций	1.96	1.88

Качественно получилось как на Fig. 3a. При постоянном маленьком LR оценка после всплеска падает примерно до значения самого LR (у авторов до 0.2, у меня 0.19). При CLR оценка остаётся высокой и растёт вместе с LR. Количественно у меня 1-2 вместо 2-6. В формуле 8 текущий LR стоит множителем, и по кривой CLR видно, что оценка в основном за ним и следует. Поэтому я бы назвал это индикатором «LR можно ещё поднять», а не независимой оценкой оптимума. Попутно сам прогон `hf_clr`, 1cycle за 2000 итераций, то есть около 20 эпох, дал 89.5 % на тесте.

1cycle против piecewise-constant

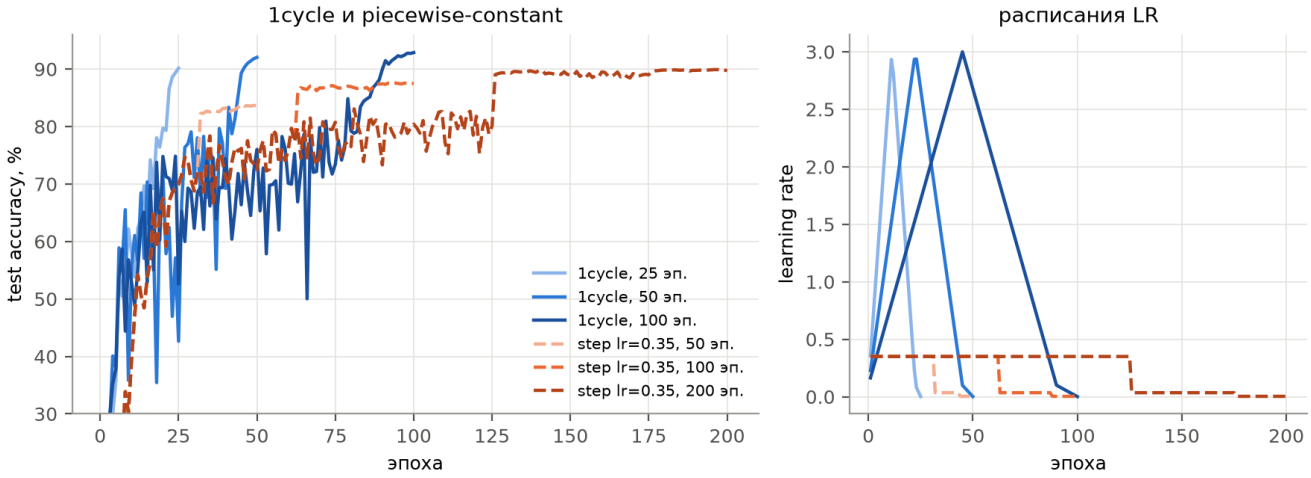


Рис. 3: Слева test accuracy по эпохам, справа расписания LR. Синие линии 1cycle 0.1 -> 3 -> 0.001 на 25 / 50 / 100 эпох, оранжевые пунктирные step с LR 0.35 на 50 / 100 / 200 эпох.

расписание	эпох	итераций	test acc, %	train acc, %	время
1cycle 0.1 -> 3 (`hf_clr`)	20.6	2 000	89.49		15 мин
1cycle 0.1 -> 3	25	2 425	**90.15**	94.2	19 мин
1cycle 0.1 -> 3	50	4 850	**92.04**	97.9	37 мин
1cycle 0.1 -> 3	100	9 700	**92.87**	99.2	74 мин
step 0.35	50	4 850	83.62	87.9	37 мин
step 0.35	100	9 700	87.49	95.8	74 мин
step 0.35	200	19 400	89.72	99.6	148 мин
step 0.1	50	4 850	88.32	96.6	37 мин
статья, CLR 0.1 -> 3, batch 1000	200	10 000	92.4		
статья, step 0.35, batch 1000	1600	80 000	91.2		

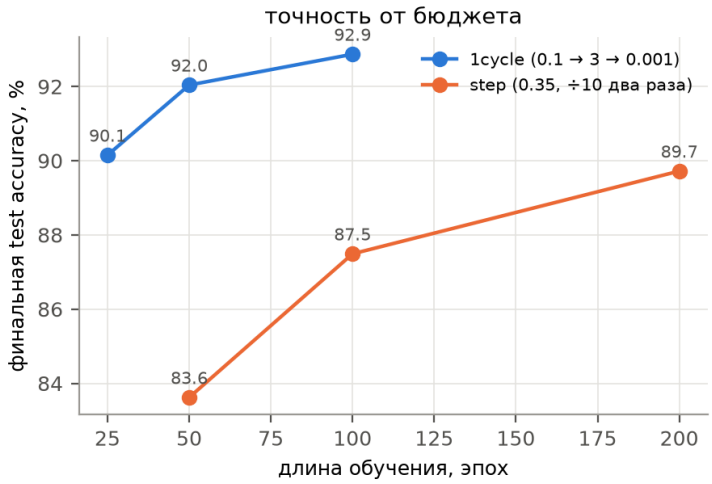


Рис. 4: Финальная точность каждого запуска в зависимости от длины обучения.

Основной результат статьи воспроизвёлся. 1cycle с максимумом 3 даёт 92.0 % за 50 эпох и 92.9 % за 100. Это на уровне чисел статьи (92.4 % за 200 эпох, 92.7 % за 400) при вдвое-вчетверо меньшем бюджете и batch 512 вместо 1000. Небольшое превышение, думаю, идёт от аугментации, инициализации Kaiming, циклического momentum и хвоста 1cycle до LR 0.001, которых у авторов не было.

Step-расписание с LR 0.35 при том же числе эпох отстаёт на 5-8 п.п.: 83.6 % за 50 эпох, 87.5 % за 100. Даже за 200 эпох оно даёт 89.7 %, что ниже 1cycle за 50 эпох. В статье для этого baseline указано 91.2 %, но после 1600 эпох. По графику точности от бюджета видно, что у step кривая растёт круто и ещё не вышла на плато, а 1cycle почти сразу выходит на 92 %. Супер-сходимость в этом смысле это именно сдвиг кривой «точность от бюджета» влево.

Форма кривой обучения совпадает с Fig. 1a. Пока LR большой, test accuracy шумит в диапазоне 50-80 %, а на спуске за последние 10-15 % обучения вырастает на 20 п.п. У step классическая картина: плато при LR 0.35 и скачок после каждого деления.

Baseline с LR 0.35 при batch 512 за 50 эпох явно недоучен, train accuracy всего 88 %. Поэтому я дополнительно прогнал обычный baseline с LR 0.1: 88.3 % за 50 эпох. Это лучше, чем 0.35, но всё равно ниже 1cycle за 25 эпох (90.2 %) и на 3.7 п.п. ниже 1cycle за те же 50 эпох. Вывод не меняется, но разрыв между 1cycle и нормально настроенным step-расписанием при равном бюджете 3-4 п.п., а не 8.

Generalization gap

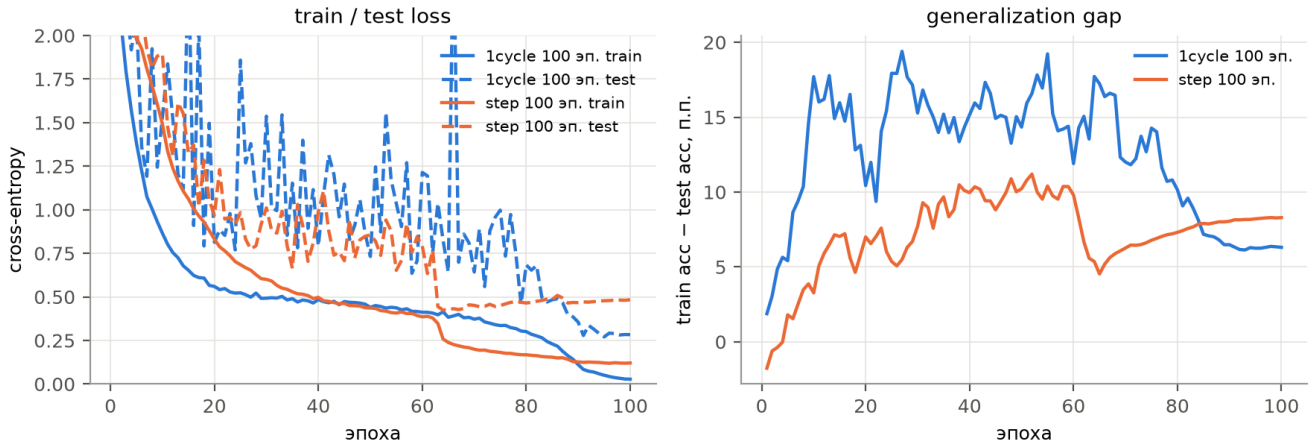


Рис. 5: Слева train и test loss для запусков на 100 эпох, справа разница train – test accuracy, сглажена по 5 эпохам.

В конце обучения разрыв train – test у 1cycle 6.3 п.п., у step 8.3 п.п., хотя 1cycle доводит train loss до 0.03, а step только до 0.12; test loss соответственно 0.28 и 0.48. Это в пользу тезиса, что большой LR в середине обучения работает как регуляризация. В середине обучения разрыв у 1cycle наоборот вдвое больше, 15-20 п.п. против 8-12, но там сравнивается train accuracy, усреднённая по эпохе, с мгновенной test accuracy при очень шумных весах, так что этому числу я бы не доверял.

Мало данных

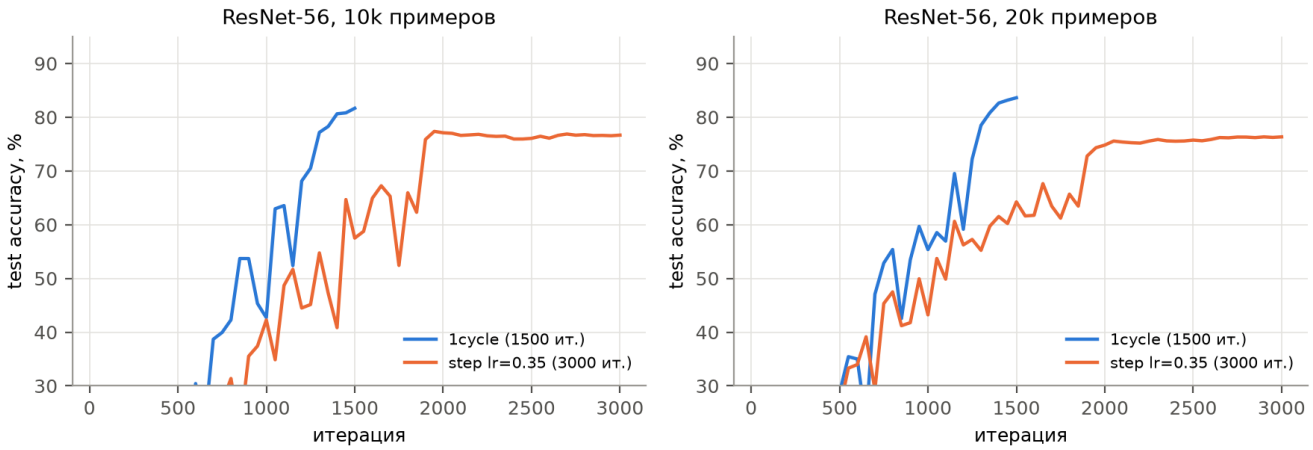


Рис. 6: Подвыборки CIFAR-10 по 10 и 20 тысяч примеров с равным числом примеров на класс. 1cycle 0.1 -> 3 за 1500 итераций против step с LR 0.35 за 3000 итераций, batch 512.

примеров	расписание	итераций	эпох	test acc, %	train acc, %
10 000	1cycle	1 500	79	**81.66**	95.6
10 000	step 0.35	3 000	158	76.66	97.5

примеров	расписание	итераций	эпох	test acc, %	train acc, %
20 000	1cycle	1 500	38	**83.62**	89.8
20 000	step 0.35	3 000	77	76.34	84.8
статья, 10 000	CLR / step	10k / 80k		80.6 / 71.4	
статья, 20 000	CLR / step	10k / 80k		87.9 / 82.7	

1cycle при вдвое меньшем числе итераций выигрывает 5.0 п.п. на 10 тысячах примеров и 7.3 п.п. на 20 тысячах. На 10 тысячах мой результат 81.7 % даже выше 80.6 % из статьи, хотя у авторов было в 7 раз больше итераций. Направление эффекта то же, что в Table 1: с 1cycle сеть на 10 тысячах примеров даёт столько же, сколько step на 20 тысячах. Расхождения у step, про которое пишут авторы, я не видел, но его точность при LR 0.35 прыгает в диапазоне 30-75 % до первого деления LR.

Оговорка: здесь 1cycle не выигрывает «в разы» по эпохам. На 10 тысячах примеров 1500 итераций это уже 79 эпох, и обе модели доходят до train accuracy выше 95 %. Выигрыш в качестве найденного решения, а не только в скорости.

Weight decay, максимальный LR, Adam

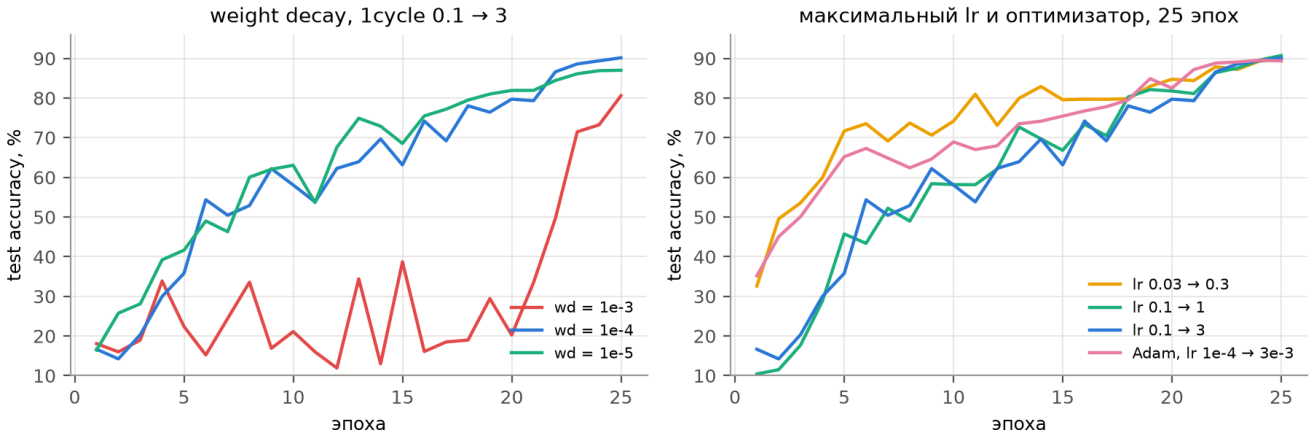


Рис. 7: 1cycle на 25 эпох. Слева weight decay при максимуме LR 3, справа максимум LR 0.3 / 1 / 3 с SGD и Adam с 1cycle по LR (1e-4 -> 3e-3) и по β_1 (0.95 -> 0.85).

запуск	WD	LR	test acc, %	train acc, %	test acc в середине цикла (эпохи 9-16), %
1cycle	10^{-3}	0.1 -> 3	80.59	80.6	21 (минимум 12)
1cycle	10^{-4}	0.1 -> 3	**90.15**	94.2	63
1cycle	10^{-5}	0.1 -> 3	87.01	92.0	67
1cycle	10^{-4}	0.1 -> 1	**90.71**	95.4	65
1cycle	10^{-4}	0.03 -> 0.3	89.70	96.3	78

запуск	WD	LR	test acc, %	train acc, %	test acc в середине цикла (эпохи 9-16), %
Adam + 1cycle	10^{-4}	10^{-4} -> $3 \cdot 10^{-3}$	89.36	95.0	71
step, 50 эпох	10^{-4}	0.1	88.32	96.6	

Weight decay повторился в точности как на Fig. 12b. С WD 10^{-3} на пике LR сеть почти не различает классы (test accuracy 12-20 %, test loss больше 3) и за оставшиеся эпохи добирается только до 80.6 %. Train accuracy при этом тоже 80.6 %, то есть это недообучение от избытка регуляризации. WD 10^{-4} оптимален, при 10^{-5} результат хуже на 3 п.п., причём train accuracy тоже ниже. Так что «баланс регуляризации» из статьи подтверждается с обеих сторон.

С максимальным LR результат расходится с Table 6. У авторов при batch 1000 максимум 3 лучше, чем 1 (92.4 % против 91.3 %). У меня при batch 512 и 25 эпохах максимум 1 даёт 90.7 % против 90.2 % при 3, а осторожный цикл 0.03 -> 0.3 даёт 89.7 %, при том что в середине обучения его кривая гораздо выше (78 % против 63 %). Разница 0.5-1 п.п. между вариантами это уровень шума одного запуска, сид у всех один. Я бы сделал такие выводы: важен сам цикл «разгон, большой LR, длинный спуск», а не конкретное значение максимума; значение 3 в статье связано с batch 1000, при меньшем batch оптимум ниже; чем больше максимум, тем глубже провал в середине и тем сильнее «супер-сходимость» в смысле формы кривой, но финальная точность от этого не растёт.

С Adam утверждение статьи у меня не подтвердилось. Adam с 1cycle по LR и β_1 даёт 89.4 % за 25 эпох, на 0.8 п.п. хуже SGD с тем же расписанием, и кривая у него такой же формы: шумная середина, резкий рост на спуске. Понятно, что «большой LR» для Adam это $3 \cdot 10^{-3}$, а не 3, так что речь о переносимости расписания, а не о величине шага.

Выводы

Основное в статье повторилось. ResNet-56 с batch norm действительно учится при LR до 3, один цикл 0.1 -> 3 за 50 эпох даёт 92.0 %, что на уровне чисел статьи, и кривая обучения выглядит так же, как на Fig. 1a: шум на пике LR и резкий рост на спуске. Step-расписание с LR 0.35 при 50 эпохах отстаёт на 8 п.п. и за 200 эпох всё ещё не догоняет 1cycle за 50; с обычным LR 0.1 разрыв при равном бюджете 3-4 п.п. На малых подвыборках 1cycle выигрывает 5-7 п.п. при вдвое меньшем числе итераций. Weight decay ведёт себя как в статье: 10^{-3} ломает супер-сходимость, 10^{-4} оптимален, 10^{-5} уже хуже. Разрыв train - test в конце обучения у 1cycle меньше, чем у step, а оценка LR по формуле 8 качественно повторяет Fig. 3a.

Есть и расхождения. Максимальный LR 3 не обязателен: при batch 512 максимум 1 даёт не хуже, и даже цикл 0.03 -> 0.3 отстаёт всего на 0.5 п.п. Большой LR в статье, по-моему, в первую очередь следствие batch 1000. Adam с 1cycle у меня работает, 89.4 % против 90.2 % у SGD, хотя авторы пишут, что с Adam эффекта получить не удалось. Ускорение «на порядок» верно только в итерациях при batch 1000, в эпохах у авторов это 200 против 1600, а у меня выигрыш примерно четырёхкратный. Рост train loss при падающем test loss в range test (Fig. 8a) я не увидел, и оценка по формуле 8 даёт 1-2 вместо 2-6, причём

в основном следует за текущим LR.

Для себя я вынес вот что: ценность статьи не в числе 3, а в схеме «разгон, большой LR, длинный спуск почти до нуля» и в мысли, что большой LR это регуляризация, под которую остальную регуляризацию нужно ослаблять. В таком виде результат стабильно повторяется на ноутбуке за 20-70 минут. Все числа получены по одному запуску с seed 0, поэтому разницу меньше 1 п.п. я считаю незначимой.

Приложение: все запуски

Полная таблица в `results/summary.md`, логи по эпохам в `results/<run>/log.csv`.

запуск	режим	LR	WD	batch	примеров	итераций	эпох	acc, %	мин
range_bs128	range	0 -> 3	1e-4	128	50 000	3 900	10	(макс. 65.8)	10.5
range_bs512	range	0 -> 3	1e-4	512	50 000	970	10	(макс. 71.3)	9.9
range_bs1000	range	0 -> 3	1e-4	1000	50 000	500	10	(макс. 62.6)	10.5
hf_const	const	0.1	1e-4	512	50 000	300	3.1	41.8	2.2
hf_clr	1cycle	0.1 -> 3	1e-4	512	50 000	2 000	20.6	89.49	14.8
1cycle_e25	1cycle	0.1 -> 3	1e-4	512	50 000	2 425	25	90.15	18.5
1cycle_e50	1cycle	0.1 -> 3	1e-4	512	50 000	4 850	50	92.04	36.7
1cycle_e100	1cycle	0.1 -> 3	1e-4	512	50 000	9 700	100	92.87	73.8
step_e50	step	0.35	1e-4	512	50 000	4 850	50	83.62	36.6
step_e100	step	0.35	1e-4	512	50 000	9 700	100	87.49	74.0
step_e200	step	0.35	1e-4	512	50 000	19 400	200	89.72	145.2
step_e50_lr0.1	step	0.1	1e-4	512	50 000	4 850	50	88.32	38.1
sub10k_1cycle	1cycle	0.1 -> 3	1e-4	512	10 000	1 500	79	81.66	11.5
sub10k_step	step	0.35	1e-4	512	10 000	3 000	158	76.66	22.7
sub20k_1cycle	1cycle	0.1 -> 3	1e-4	512	20 000	1 500	38	83.62	11.1
sub20k_step	step	0.35	1e-4	512	20 000	3 000	77	76.34	22.9
1cycle_e25_wd1e-3	1cycle	0.1 -> 3	1e-3	512	50 000	2 425	25	80.59	18.5
1cycle_e25_wd1e-5	1cycle	0.1 -> 3	1e-5	512	50 000	2 425	25	87.01	18.2
1cycle_e25_lr1	1cycle	0.1 -> 1	1e-4	512	50 000	2 425	25	90.71	18.8
1cycle_e25_lr0.3	1cycle	0.03 -> 0.3	1e-4	512	50 000	2 425	25	89.70	19.3
adam_1cycle_e25	1cycle, Adam	1e-4 -> 3e-3	1e-4	512	50 000	2 425	25	89.36	19.4

Всего около 10 часов счёта на MacBook Air M5, PyTorch 2.14, MPS, fp32. Чтобы повторить: `python prepare_data.py`, затем `./run_all.sh` и `python plots.py`; на колабе то же самое в `hw2_colab.ipynb`.